Extending SGLang Technically

Deep Dive into SGLang — Chapter 35

Yin Yang

Foundation Books Project

Where this chapter sits

This is the last chapter, and it turns "I wrote model code that generates tokens" into "I added a supported SGLang feature."

- A contributor adds `LlamaWrapper`, points `./llama\_ckpt` at it, and a local script prints tokens.
- The same patch can still fail when the *server* streams a response: serving needs registry resolution, request state, cache ownership, logits handling, backend metadata, and a mode test.
- Every mechanism from the whole book — registry, forward batch, KV ownership, attention backends, kernels, sampling — reappears here as an *extension surface* a new feature must cross.

The claim: extension code becomes SGLang support only when it survives the runtime path it claims.

Roadmap

The integration record

Two running traces

Readiness and review

The integration record

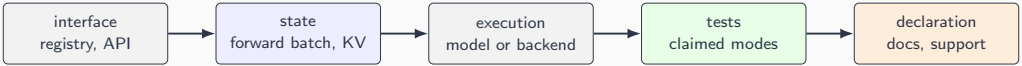
One feature, five obligations

Definition (Extension integration record)
$E = (I, \Sigma, \mathcal{V}, \mathcal{D}, \mathcal{M})$: interfaces, state invariants, validation evidence, declarations, and modes.

- A model file without registry support lacks I.
- A new attention path without cache/position invariants lacks Σ.
- A kernel without local and runtime tests lacks $\mathcal{V}$.
- A supported model without support docs lacks $\mathcal{D}$.
- A public claim without an explicit mode ledger lacks $\mathcal{M}$.

The record is deliberately stricter than a code checklist: it is not done until all five fields move together.

The checklist crosses more than one file



An extension integration record follows the feature through interfaces, state invariants, execution paths, tests, and user-facing declarations — not just the new implementation file.

Two running traces

The chapter carries two concrete traces

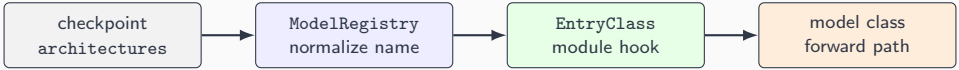
`LlamaWrapper` a decoder-only *model* extension for a Llama 3.1 8B checkpoint. Claims offline Engine generation plus OpenAI-compatible server generation.

`bmm_fp8` an FP8 batched matmul *kernel* extension: device code, C++ interface, torch-library schema, Python wrapper, test.

- The model trace shows registry → forward batch → logits → mode test.
- The kernel trace shows Python API → schema → CUDA binding → reference oracle.

Both are intentionally small — small enough that every runtime handoff is visible, and none can be skipped.

Registry resolution comes first



If the architecture name does not resolve, the extension never reaches serving. The name in the checkpoint, the class exported as `EntryClass`, and the registry mapping must all agree.

Readiness and review

A mode is claimed only after its evidence exists

Definition (Validation anchor)
The first runtime handoff whose owned state and oracle can prove or reject one extension claim — state owner, comparison/invariant, and rejection path.

- Offline mode: registry/load test + one-batch output comparison.
- Server mode: streaming/non-streaming + `/v1/chat/completions` test.
- Kernel: local operator test + one runtime consumer.
- If the oracle is missing, the honest result is a *narrower* mode declaration — not a larger table of touched files.

The mode ledger $\mathcal{M}$ records both claimed and intentionally rejected modes, so docs cannot advertise a path tests do not cover.

What to carry forward

1. Extending SGLang means proving new code **survives the runtime path it claims** — not that a local script prints tokens.
2. The **integration record** $E = (I, \Sigma, \mathcal{V}, \mathcal{D}, \mathcal{M})$ is the closing checklist; a missing field means the review is not finished.
3. Resolution starts at the **registry**: name, `EntryClass`, and mapping must agree.
4. Every claimed mode needs a **validation anchor** — a state owner and an oracle at the first handoff that can falsify it.
5. **Alignment** across claim, interface, test, and documentation is the whole job.

This closes Deep Dive into SGLang. The through-line held from Chapter 2 to here: every concept has a home in the source, and a feature is real only when its claim, state, tests, and docs name one system.